

DS 专题选讲

2026.8 讲题

Cyril

2026-08-25



安徽师范大学附属中学

THE HIGH SCHOOL AFFILIATED TO ANHUI NORMAL UNIVERSITY

Full Presentation

Abstract

这是一份 DS 选讲题单。可能涵盖了线段树、树链剖分、分块等数据结构。

大部分题目来自于学长 **xex**。

只有 CF2182G 是我自己做出来的题目，用时大约 2.5h，并没有什么参考意义。

PPT 上的题意均经过简化，可能需要从原始题意浅层转化得到。

题目描述上方的链接可以直接跳转到原题。

没有提到强制在线的题目默认可以考虑离线。

CF2182G
○○○○○○○

CF1572F
○○○○

LG P8861
○○○○○○

LG P15397
○○○○○

LG P12485
○○○○

LG P8987
○○○○○

CF1685E
○○○○○○

Outline

- **CF2182G**
- **CF1572F**
- **LG P8861**
- **LG P15397**
- **LG P12485**
- **LG P8987**
- **CF1685E**

Outline

- **CF2182G**
- CF1572F
- LG P8861
- LG P15397
- LG P12485
- LG P8987
- CF1685E

Statement

CF2182G Short Garland

给定一颗 n 个节点的树，以 1 为根。

定义一个好的排列为，将排列首尾相接成环后，相邻两个数所代表的节点的距离不超过 K 。

计算好的 DFS 序的个数。

$k < n \leq 3 \times 10^5$.

Hints

1. 考虑设计一个 $\mathcal{O}(n^2)$ 的 DP 状态。

Hints

1. 考虑设计一个 $\mathcal{O}(n^2)$ 的 DP 状态。
2. 考虑使用数据结构优化至线性。

Solution

以下“深度”均为相对当前讨论子树的深度。

设计状态 $f_{u,i}$ 表示 DFS 完节点 u 子树结束时所在点深度为 i 的方案数。

Solution

以下“深度”均为相对当前讨论子树的深度。

设计状态 $f_{u,i}$ 表示 DFS 完节点 u 子树结束时所在点深度为 i 的方案数。

考虑我们可以枚举最后进入哪一颗子树，然后从那颗子树的状态转移过来。

Solution

以下“深度”均为相对当前讨论子树的深度。

设计状态 $f_{u,i}$ 表示 DFS 完节点 u 子树结束时所在点深度为 i 的方案数。

考虑我们可以枚举最后进入哪一颗子树，然后从那颗子树的状态转移过来。

具体来说如果有 m 个子树，枚举最后进入第 j 个子树，那么剩余的子树可以以任意顺序进入和退出，但是需要满足相邻点距离不超过 K 的限制。

Solution

以下“深度”均为相对当前讨论子树的深度。

设计状态 $f_{u,i}$ 表示 DFS 完节点 u 子树结束时所在点深度为 i 的方案数。

考虑我们可以枚举最后进入哪一颗子树，然后从那颗子树的状态转移过来。

具体来说如果有 m 个子树，枚举最后进入第 j 个子树，那么剩余的子树可以以任意顺序进入和退出，但是需要满足相邻点距离不超过 K 的限制。

容易发现，如果从一个子树 v_1 经过根节点 u 转移到另外一个子树 v_2 ，那么在前面这颗子树 v_1 当中访问的最深节点的深度不能超过 $K - 2$ 。

Solution

以下“深度”均为相对当前讨论子树的深度。

设计状态 $f_{u,i}$ 表示 DFS 完节点 u 子树结束时所在点深度为 i 的方案数。

考虑我们可以枚举最后进入哪一颗子树，然后从那颗子树的状态转移过来。

具体来说如果有 m 个子树，枚举最后进入第 j 个子树，那么剩余的子树可以以任意顺序进入和退出，但是需要满足相邻点距离不超过 K 的限制。

容易发现，如果从一个子树 v_1 经过根节点 u 转移到另外一个子树 v_2 ，那么在前面这颗子树 v_1 当中访问的最深节点的深度不能超过 $K - 2$ 。

所以我们能够得出转移：

Solution

$$f_{u,i} = \sum_v f_{v,i-1} (|\text{ch}_u| - 1)! \prod_{v' \neq v} \sum_{j=0}^{K-2} f_{v',j}$$

Solution

$$f_{u,i} = \sum_v f_{v,i-1} (|\text{ch}_u| - 1)! \prod_{v' \neq v} \sum_{j=0}^{K-2} f_{v',j}$$

考虑将其写成前缀和形式。

Solution

$$f_{u,i} = \sum_v f_{v,i-1} (|\text{ch}_u| - 1)! \prod_{v' \neq v} \sum_{j=0}^{K-2} f_{v',j}$$

考虑将其写成前缀和形式。

$$sf_{u,i} = \sum_v sf_{v,i-1} (|\text{ch}_u| - 1)! \prod_{v' \neq v} sf_{v',K-2}$$

Solution

$$f_{u,i} = \sum_v f_{v,i-1} (|\text{ch}_u| - 1)! \prod_{v' \neq v} \sum_{j=0}^{K-2} f_{v',j}$$

考虑将其写成前缀和形式。

$$\text{sf}_{u,i} = \sum_v \text{sf}_{v,i-1} (|\text{ch}_u| - 1)! \prod_{v' \neq v} \text{sf}_{v',K-2}$$

最终所求的答案也就是 $\text{sf}_{1,K}$ 。

Solution

$$f_{u,i} = \sum_v f_{v,i-1} (|\text{ch}_u| - 1)! \prod_{v' \neq v} \sum_{j=0}^{K-2} f_{v',j}$$

考虑将其写成前缀和形式。

$$\text{sf}_{u,i} = \sum_v \text{sf}_{v,i-1} (|\text{ch}_u| - 1)! \prod_{v' \neq v} \text{sf}_{v',K-2}$$

最终所求的答案也就是 $\text{sf}_{1,K}$ 。

显然这个时间复杂度时可以优化到 $\mathcal{O}(n^2)$ 的。无法通过本题。

Solution

$$f_{u,i} = \sum_v f_{v,i-1} (|\text{ch}_u| - 1)! \prod_{v' \neq v} \sum_{j=0}^{K-2} f_{v',j}$$

考虑将其写成前缀和形式。

$$\text{sf}_{u,i} = \sum_v \text{sf}_{v,i-1} (|\text{ch}_u| - 1)! \prod_{v' \neq v} \text{sf}_{v',K-2}$$

最终所求的答案也就是 $\text{sf}_{1,K}$ 。

显然这个时间复杂度时可以优化到 $\mathcal{O}(n^2)$ 的。无法通过本题。

考虑套路性的使用长链剖分进行优化。

Solution

考虑套路性的使用长链剖分进行优化。

$$sf_{u,i} = (|\text{ch}_u| - 1)! \sum_v sf_{v,i-1} \prod_{v' \neq v} sf_{v',K-2}$$

Solution

考虑套路性的使用长链剖分进行优化。

$$sf_{u,i} = (|ch_u| - 1)! \sum_v sf_{v,i-1} \prod_{v' \neq v} sf_{v',K-2}$$

发现我们转移的时候，可以直接复制重儿子的 f 数组，乘上一个系数

$$\prod_{v' \neq hson_u} sf_{v',K-2}.$$

对于轻儿子，直接暴力将其有效项的贡献加进去，即前深度项。后面的因为 $sf_{v,i}$ 都是不变的，所以可以整体加。

转移结束还要在外面乘上一个系数 $(|ch_u| - 1)!$.

对于叶子节点，有 $sf(u, 0) = 1$.

Solution

考虑套路性的使用长链剖分进行优化。

$$sf_{u,i} = (|ch_u| - 1)! \sum_v sf_{v,i-1} \prod_{v' \neq v} sf_{v',K-2}$$

发现我们转移的时候，可以直接复制重儿子的 f 数组，乘上一个系数

$$\prod_{v' \neq hson_u} sf_{v',K-2}.$$

对于轻儿子，直接暴力将其有效项的贡献加进去，即前深度项。后面的因为 $sf_{v,i}$ 都是不变的，所以可以整体加。

转移结束还要在外面乘上一个系数 $(|ch_u| - 1)!$ 。

对于叶子节点，有 $sf(u, 0) = 1$ 。

所以我们需要维护一个支持整体乘、后缀加、单点修改的顺序数据结构。

Solution

所以我们需要维护一个支持整体乘、后缀加、单点修改的顺序数据结构。

考虑使用 `(*arr, add, mul)` 进行维护。

Solution

所以我们需要维护一个支持整体乘、后缀加、单点修改的顺序数据结构。

考虑使用 `(*arr, add, mul)` 进行维护。

我们对于整体乘、后缀加转化为打上 `mul, add` 的 `tag`，然后单点修改直接将数组中的值更换。

具体来说如果我们要 $\text{reval}a_i \leftarrow v$ ，让 $a_i \leftarrow x$ 使得 $\text{mul}x + \text{add} = v$ 即可。

Solution

所以我们需要维护一个支持整体乘、后缀加、单点修改的顺序数据结构。

考虑使用 $(*arr, add, mul)$ 进行维护。

我们对于整体乘、后缀加转化为打上 mul, add 的 tag ，然后单点修改直接将数组中的值更换。

具体来说如果我们要 $real a_i \leftarrow v$ ，让 $a_i \leftarrow x$ 使得 $mulx + add = v$ 即可。

同时我们需要注意，如果乘上的系数为 0，我们需要将数组清 0，因为在有 add 的情况下，无法找到一个合适的方式去整体操作。

Solution

所以我们需要维护一个支持整体乘、后缀加、单点修改的顺序数据结构。

考虑使用 $(*arr, add, mul)$ 进行维护。

我们对于整体乘、后缀加转化为打上 mul, add 的 tag ，然后单点修改直接将数组中的值更换。

具体来说如果我们要 $real a_i \leftarrow v$ ，让 $a_i \leftarrow x$ 使得 $mulx + add = v$ 即可。

同时我们需要注意，如果乘上的系数为 0，我们需要将数组清 0，因为在有 add 的情况下，无法找到一个合适的方式去整体操作。

所以我们再加一个全局清空的 tag ，具体来说我们维护后缀为 0 的长度，然后直接假设后面的 a_i 存的都是 0 就行了。

具体实现的话后面的这个全局清空可以暴力去清空数组，因为 CF 并没有去卡。^_^

Outline

- CF2182G
- **CF1572F**
- LG P8861
- LG P15397
- LG P12485
- LG P8987
- CF1685E

Statement

CF1572F Stations

现在有 n 座塔，有高度 h_i 和广播范围 w_i .

定义塔 i 能向塔 j 发送信息，当且仅当 $j \in [i, w_i]$ 且对于 $k \in (i, j]$ 都有 $h_k < h_i$ ，即 h_i 是 $h_{i\dots j}$ 中的严格最大值。

定义 b_i 为能够发送信息到塔 i 的塔的数量。

初始有 $\forall i \in [1, n], h_i = 0, w_i = i$.

接下来有 q 个事件，两种操作：

1. (c, g) 修改: $h_c \leftarrow \max_{i=1}^n \{h_i\} + 1, w_c \leftarrow g$
2. (l, r) 询问: $\sum_{i=l}^r b_i$

$n, q, g \leq 2 \times 10^5$.

Hints

1. 考虑维护每个节点实际的广播范围 $[i, f_i]$.

Solution

本题的 DS 比较板，可以考虑记住这个技巧。

Solution

本题的 DS 比较板，可以考虑记住这个技巧。

每次修改操作 (c, g) 对于 f 的影响实际上是 $\forall i \in [1, c - 1], f_i \leftarrow \min\{f_i, c - 1\}$ 和 $f_c \leftarrow g$.

Solution

本题的 DS 比较板，可以考虑记住这个技巧。

每次修改操作 (c, g) 对于 f 的影响实际上是 $\forall i \in [1, c - 1], f_i \leftarrow \min\{f_i, c - 1\}$ 和 $f_c \leftarrow g$.

但是如何维护 b_i 呢？

Solution

本题的 DS 比较板，可以考虑记住这个技巧。

每次修改操作 (c, g) 对于 f 的影响实际上是 $\forall i \in [1, c - 1], f_i \leftarrow \min\{f_i, c - 1\}$ 和 $f_c \leftarrow g$.

但是如何维护 b_i 呢？

考虑我们进行 SegBeats 的过程，每次只会将一些相同的 cnt 个 f_x 修改为 $f_{x'}$ ，那么 $b_{f_{x'}+1 \dots f_x}$ 都会减去 cnt 。

Solution

本题的 DS 比较板，可以考虑记住这个技巧。

每次修改操作 (c, g) 对于 f 的影响实际上是 $\forall i \in [1, c - 1], f_i \leftarrow \min\{f_i, c - 1\}$ 和 $f_c \leftarrow g$.

但是如何维护 b_i 呢？

考虑我们进行 SegBeats 的过程，每次只会将一些相同的 cnt 个 f_x 修改为 $f_{x'}$ ，那么 $b_{f_{x'}+1 \dots f_x}$ 都会减去 cnt 。

这题就做完了。维护两颗线段树，一颗 SegBeats 可以 chkmin ，一颗区间加区间求和线段树即可。

CF2182G
○○○○○○○

CF1572F
○○○○

LG P8861
●○○○○○

LG P15397
○○○○○

LG P12485
○○○○

LG P8987
○○○○○

CF1685E
○○○○○○○

Outline

- CF2182G
- CF1572F
- **LG P8861**
- LG P15397
- LG P12485
- LG P8987
- CF1685E

Statement

P8861 线段

有一个初始为空的线段集，你需要处理 q 组询问，每组询问的格式为如下三种之一：

1. 加入一条新线段 $[l, r]$ 。
2. 将线段集里所有与 $[l, r]$ 相交的线段修改为其与 $[l, r]$ 的交。
3. 求出线段集里所有与 $[l, r]$ 相交的线段与 $[l, r]$ 的交的长度和。

一条线段 $[a, b]$ 的长度为 $b - a$ 。在本题中，线段可能退化为单点。

注意：你需要在线地处理每一组询问。

记 q_1, q_2, q_3 表示三种询问的数量。

有 $n \leq 2 \times 10^5, q_1, q_2 \leq 10^5, q_3 \leq 3 \times 10^5$ 。

Hints

1. 考虑如果只有操作 1 和 3, 怎么做。

Hints

1. 考虑如果只有操作 1 和 3, 怎么做。
2. 考虑特殊性质 $l_i \leq 10^5 \leq r_i$ 这一档, 怎么做。

Hints

1. 考虑如果只有操作 1 和 3, 怎么做。
2. 考虑特殊性质 $l_i \leq 10^5 \leq r_i$ 这一档, 怎么做。
3. 考虑使用类似于猫树分治的思路去维护所有的区间。

Solution

本题具有极大的启发意义。建议学习。

Solution

本题具有极大的启发意义。建议学习。

我们先把所有的区间假设为点的左闭右开，这样方便处理。

如果只有操作 1 和操作 3, 不妨把操作 1 看作是区间加，操作 3 看作是区间求和。

Solution

本题具有极大的启发意义。建议学习。

我们先把所有的区间假设为点的左闭右开，这样方便处理。

如果只有操作 1 和操作 3, 不妨把操作 1 看作是区间加，操作 3 看作是区间求和。

有了这个核心观察之后，本题我们所有的操作 3 都将在一个支持区间求和的 DS 上查询。

Solution

本题具有极大的启发意义。建议学习。

我们先把所有的区间假设为点的左闭右开，这样方便处理。

如果只有操作 1 和操作 3, 不妨把操作 1 看作是区间加，操作 3 看作是区间求和。

有了这个核心观察之后，本题我们所有的操作 3 都将在一个支持区间求和的 DS 上查询。

我们再考虑如果 l, r 被分成左右两块应该怎么做。

Solution

本题具有极大的启发意义。建议学习。

我们先把所有的区间假设为点的左闭右开，这样方便处理。

如果只有操作 1 和操作 3, 不妨把操作 1 看作是区间加，操作 3 看作是区间求和。

有了这个核心观察之后，本题我们所有的操作 3 都将在一个支持区间求和的 DS 上查询。

我们再考虑如果 l, r 被分成左右两块应该怎么做。

显然此时所有的区间都经过 Mid , 这就启发我们对于左边的左端点统一处理，右边的右端点统一处理。稍加思考我们发现，因为永远经过 Mid , 所以左边端点匹配哪个右边端点并不重要，因为我们最后只需要区间求和就行了。

于是我们给出解法：如果与 $[l, r)$ 取交，将左边端点与 l 取最大，右边端点与 r 取最小，使用上一题的策略就可以维护区间和了。

CF2182G
○○○○○○○

CF1572F
○○○○

LG P8861
○○○○●○

LG P15397
○○○○○

LG P12485
○○○○

LG P8987
○○○○○

CF1685E
○○○○○○

Solution

现在考虑推广这个优美的做法。

Solution

现在考虑推广这个优美的做法。

我们使用类似于猫树的思想，即在线段树上的一个分治区间，维护所有跨过区间中点的线段。

Solution

现在考虑推广这个优美的做法。

我们使用类似于猫树的思想，即在线段树上的一个分治区间，维护所有跨过区间中点的线段。

显然插入线段不用我多说，一条线段一定唯一属于一个分治区间。

Solution

现在考虑推广这个优美的做法。

我们使用类似于猫树的思想，即在线段树上的一个分治区间，维护所有跨过区间中点的线段。

显然插入线段不用我说，一条线段一定唯一属于一个分治区间。

如果我们现在要将所有线段与 $[L, R)$ 取交，那么我们就按照线段树结构分治下去做。

具体来说，如果当前分治区间为 $[l, r)$ 且中点为 mid ，而且 $[L, R)$ 包含 mid ，我们就将所有左端点 chkmax 而右端点 chkmin 。

如果 $[L, R)$ 不包含 mid ，我们发现 $[l, r)$ 区间内与 $[L, R)$ 交的线段都“降级”了，也就是不可能再经过 mid 了，对于这些线段我们直接暴力给他降级，拿出来取交然后再塞到更小的分治区间里面去。

Solution

考虑刚刚这个口胡做法的时间复杂度。首先我们分析降级操作，每一条线段只会变短，最多被降级 $\mathcal{O}(\log n)$ 次。而对于 `chkmin` `chkmax` 操作，如果能把相同端点合起来维护的话，最多只有 $\mathcal{O}(n)$ 种不同的端点，每次 `chk` 几个就能够合几个，所以也是最多合并不同端点 $\mathcal{O}(n)$ 次。

Solution

考虑刚刚这个口胡做法的时间复杂度。首先我们分析降级操作，每一条线段只会变短，最多被降级 $\mathcal{O}(\log n)$ 次。而对于 `chkmin` `chkmax` 操作，如果能把相同端点合起来维护的话，最多只有 $\mathcal{O}(n)$ 种不同的端点，每次 `chk` 几个就能够合几个，所以也是最多合并不同端点 $\mathcal{O}(n)$ 次。

后面的故事就都是细节了，展开说大家肯定不太愿意听。但是我还是提几点：

Solution

考虑刚刚这个口胡做法的时间复杂度。首先我们分析降级操作，每一条线段只会变短，最多被降级 $\mathcal{O}(\log n)$ 次。而对于 `chkmin` `chkmax` 操作，如果能把相同端点合起来维护的话，最多只有 $\mathcal{O}(n)$ 种不同的端点，每次 `chk` 几个就能够合几个，所以也是最多合并不同端点 $\mathcal{O}(n)$ 次。

后面的故事就都是细节了，展开说大家肯定不太愿意听。但是我还是提几点：

1. 左右端点集合的维护可以考虑使用并查集 + vector，即对于一个位置 p 维护线段端点为 p 的线段编号集合，启发式合并即可。为什么要维护线段端点？因为“降级”操作需要知道线段长啥样啊，同时线段数量也是在 DS 上更新贡献的基础。

Solution

考虑刚刚这个口胡做法的时间复杂度。首先我们分析降级操作，每一条线段只会变短，最多被降级 $\mathcal{O}(\log n)$ 次。而对于 `chkmin chkmax` 操作，如果能把相同端点合起来维护的话，最多只有 $\mathcal{O}(n)$ 种不同的端点，每次 `chk` 几个就能够合几个，所以也是最多合并不同端点 $\mathcal{O}(n)$ 次。

后面的故事就都是细节了，展开说大家肯定不太愿意听。但是我还是提几点：

1. 左右端点集合的维护可以考虑使用并查集 + vector，即对于一个位置 p 维护线段端点为 p 的线段编号集合，启发式合并即可。为什么要维护线段端点？因为“降级”操作需要知道线段长啥样啊，同时线段数量也是在 DS 上更新贡献的基础。
2. `chkmin chkmax` 考虑使用优先队列，直接暴力合并时间也是对的， $\log^2$ 级别。
3. 把一条线段拎出来之后，不一定要立刻删除，而是改为每次更新判断线段是不是确实属于这个分治区间的，不是的话跳过就行。这样空间 $\log$ ，但是无所谓了。

Solution

考虑刚刚这个口胡做法的时间复杂度。首先我们分析降级操作，每一条线段只会变短，最多被降级 $\mathcal{O}(\log n)$ 次。而对于 `chkmin chkmax` 操作，如果能把相同端点合起来维护的话，最多只有 $\mathcal{O}(n)$ 种不同的端点，每次 `chk` 几个就能够合几个，所以也是最多合并不同端点 $\mathcal{O}(n)$ 次。

后面的故事就都是细节了，展开说大家肯定不太愿意听。但是我还是提几点：

1. 左右端点集合的维护可以考虑使用并查集 + vector，即对于一个位置 p 维护线段端点为 p 的线段编号集合，启发式合并即可。为什么要维护线段端点？因为“降级”操作需要知道线段长啥样啊，同时线段数量也是在 DS 上更新贡献的基础。
2. `chkmin chkmax` 考虑使用优先队列，直接暴力合并时间也是对的， $\log^2$ 级别。
3. 把一条线段拎出来之后，不一定要立刻删除，而是改为每次更新判断线段是不是确实属于这个分治区间的，不是的话跳过就行。这样空间 $\log$ ，但是无所谓了。

具体实现的话可以这么说，糟搞都能过，不卡常。细节想明白应该是不需要调试的。

CF2182G
○○○○○○○

CF1572F
○○○○

LG P8861
○○○○○○○

LG P15397
●○○○○

LG P12485
○○○○

LG P8987
○○○○○

CF1685E
○○○○○○○

Outline

- CF2182G
- CF1572F
- LG P8861
- **LG P15397**
- LG P12485
- LG P8987
- CF1685E

Statement

P15397 浙江旅行团 / hangzhou

定义本题中的矩阵为 $(\min, +)$ 2×2 矩阵，满足运算 $(A \times B)_{ik} = \min(A_{ij} + B_{jk})$ 。
单位元 $\varepsilon = \begin{pmatrix} 0 & 10^9 \\ 10^9 & 0 \end{pmatrix}$ 。 定义 $\oplus$ 表示按位异或。

你需要维护 m 个 bot，版本 0 时 bot i 在位置 a_i ，矩阵 $M_i = \varepsilon$ ，评分 $r_i = 0$ 。

结算 bot i : $r_i \leftarrow r_i \oplus (A \oplus M_{i_{00}} + B \oplus M_{i_{01}} + C \oplus M_{i_{10}} + D \oplus M_{i_{11}})$, $M_i \leftarrow \varepsilon$ 。

一共有 q 组询问，三种操作：

第 i 组询问建立在版本 t_i 之上，并成立版本 i 。即需要可持久化。

1. (l, r, c) 修改：使 bot $l \sim r$ 结算，然后区间赋值 $a_{l \dots r} \leftarrow c$ 。
2. (c, V) “活动”：令所有位置在 c 的机器人的矩阵 $M_i \leftarrow M_i \times V$ 。
3. (x) 查询：对 bot x 暂时地结算（不可持久化）之后，求其评分 r_i 。

强制在线。 $n, m, q \leq 2 \times 10^5, 1 \leq a_i, c \leq n, 0 \leq A, B, C, D < 2^{30}, 0 \leq V_{ij} < 2^8$ 。

Hints

1. 考虑我们应当如何刻画一个 bot?
2. 考虑我们应当如何刻画一次“活动”?

Solution

本题对于 bot 的刻画非常具有启发性。

Solution

本题对于 bot 的刻画非常具有启发性。

我们考虑对于一个 bot, 只去管他的 (pos, time) 序列, 即在什么时刻变化到了什么位置的一个历史记录。

同时我们对于一个“活动”, 记录他发生的时间。

Solution

本题对于 bot 的刻画非常具有启发性。

我们考虑对于一个 bot, 只去管他的 (pos, time) 序列, 即在什么时刻变化到了什么位置的一个历史记录。

同时我们对于一个“活动”, 记录他发生的时间。

于是我们得到了一版暴力: 对于每一次查询, 我只需要顺着历史记录, 一个一个将“活动”统计进来就行了。

考虑现在使用线段树和倍增进行优化。

Solution

本题对于 bot 的刻画非常具有启发性。

我们考虑对于一个 bot, 只去管他的 (pos, time) 序列, 即在什么时刻变化到了什么位置的一个历史记录。

同时我们对于一个“活动”, 记录他发生的时间。

于是我们得到了一版暴力: 对于每一次查询, 我只需要顺着历史记录, 一个一个将“活动”统计进来就行了。

考虑现在使用线段树和倍增进行优化。

第一个优化就是, 我们对于一个 bot, 不需要维护整个历史记录, 因为前面的结算操作贡献是固定的, 每次迁移可以直接把结算累计进来。我们需要的仅仅是最后一个 (pos, time), 用它来做最后的暂时结算。

Solution

第二点，我们使用线段树维护 bot 的评分和二元组，tag 就设置为 (pos, time)，不断 pushdown 下去。这样每次修改就是 $\mathcal{O}(\log n)$ 的。

Solution

第二点，我们使用线段树维护 bot 的评分和二元组，tag 就设置为 (pos, time)，不断 pushdown 下去。这样每次修改就是 $\mathcal{O}(\log n)$ 的。

第三点，我们结算“活动”的贡献的时候，是询问形如从 t_0 一直到现在矩阵乘积。这个玩意让我们想到了树，所以直接动态倍增，就可以快速求解了。

Solution

第二点，我们使用线段树维护 bot 的评分和二元组，tag 就设置为 (pos, time)，不断 pushdown 下去。这样每次修改就是 $\mathcal{O}(\log n)$ 的。

第三点，我们结算“活动”的贡献的时候，是询问形如从 t_0 一直到现在矩阵乘积。这个玩意让我们想到了树，所以直接动态倍增，就可以快速求解了。

那么，本题就转化为了，线段树可持久化，树上加叶子动态维护链乘积，以及询问线段树和链乘积。

本题终了。

CF2182G
○○○○○○○

CF1572F
○○○○

LG P8861
○○○○○○

LG P15397
○○○○○

LG P12485
●○○○

LG P8987
○○○○○

CF1685E
○○○○○○

Outline

- CF2182G
- CF1572F
- LG P8861
- LG P15397
- **LG P12485**
- LG P8987
- CF1685E

Statement

P12485 [集训队互测 2024] PM 大师

对于可重集 S 定义 $\text{mex}(S)$ 表示最小的 **正整数** x 满足 $x \notin S$.

定义 $f(a) \rightarrow b$, 要求 $-1 \leq a_i \leq n$:
$$b_i = \begin{cases} a_i & a_i \neq 0 \\ \text{mex}_{j=1}^{i-1} \{b_j\} & a_i = 0 \end{cases}$$

现在给定长度为 n 的数组 a , **保证初始时 $a_i \in \{-1, 0\}$ 且数组 a 不全为 0.**

q 次操作 (x, k, y) 表示先将 $a_x \leftarrow k$, 然后求出使用 a 生成的数组 b 中 b_y 的值。

保证任意时刻为 0 的 a_i 不会被修改, 不为 0 的 a_i 不会被修改为 0.

$n, q \leq 10^6$.

Solution

本题好难。

Solution

本题好难。

我们令 S 集合表示满足 $a_i = 0$ 的所有 b_i ，即根据前缀 mex 规则生成的 b_i 。
显然有 S 集合是递增的。

Solution

本题好难。

我们令 S 集合表示满足 $a_i = 0$ 的所有 b_i ，即根据前缀 mex 规则生成的 b_i 。

显然有 S 集合是递增的。

现在我们考虑，对于 $v \in [1, n]$ ， $v \in S$ 的条件是什么。

Solution

本题好难。

我们令 S 集合表示满足 $a_i = 0$ 的所有 b_i ，即根据前缀 mex 规则生成的 b_i 。

显然有 S 集合是递增的。

现在我们考虑，对于 $v \in [1, n]$ ， $v \in S$ 的条件是什么。

条件就是，生成 v 的时候，前面不能再有 $a_i = v$ 了。

Solution

本题好难。

我们令 S 集合表示满足 $a_i = 0$ 的所有 b_i ，即根据前缀 mex 规则生成的 b_i 。

显然有 S 集合是递增的。

现在我们考虑，对于 $v \in [1, n]$ ， $v \in S$ 的条件是什么。

条件就是，生成 v 的时候，前面不能再有 $a_i = v$ 了。

刻画出来，把 a 中最小的 i 满足 $a_i = v$ 记为 p_v ， pre_i 表示 $a_{1\dots i}$ 中值为 0 的个数。

同时，这题的点睛之笔：令 s_v 表示 $\forall i \in [1, v-1], i \notin S$ 的个数。

Solution

本题好难。

我们令 S 集合表示满足 $a_i = 0$ 的所有 b_i ，即根据前缀 mex 规则生成的 b_i 。

显然有 S 集合是递增的。

现在我们考虑，对于 $v \in [1, n]$ ， $v \in S$ 的条件是什么。

条件就是，生成 v 的时候，前面不能再有 $a_i = v$ 了。

刻画出来，把 a 中最小的 i 满足 $a_i = v$ 记为 p_v ， pre_i 表示 $a_{1\dots i}$ 中值为 0 的个数。

同时，这题的点睛之笔：令 s_v 表示 $\forall i \in [1, v-1], i \notin S$ 的个数。

这样，我们只需要确认一下，在放置 $a_{p_v} = v$ 之前有几个 0，即 pre_{p_v} ，并把它与预期的 v 在第几个 0 被加入，即 $v - s_v$ ，两个数值进行比较，即可知道是否有 $v \in S$ 。

写出来就是 $v \in S \iff v - s_v \leq \text{pre}_{p_v} \iff v - s_v - \text{pre}_{p_v} \leq 0$ 。

于是转化为了我们维护每一个 v 对应的 $v - s_v - \text{pre}_{p_v}$ ，询问 a 中第 pre_i 个 0 生成的 b_i 时，线段树二分出来第 pre_i 个值 ≤ 0 的位置作为答案即可。

Solution

考虑加入修改，一次 $a_i \rightarrow a_{i'}$ 至多会将 p_{a_i} 和 $p_{a_{i'}}$ 修改，线段树上单点修改即可。

然后我们考虑如果此时两个值的状态变化了，即 $\in S \longleftrightarrow \notin S$ ，那 S 和 s 就会受到影响。线段树上的修改操作就是一段后缀 ± 1 。

Solution

考虑加入修改，一次 $a_i \rightarrow a_{i'}$ 至多会将 p_{a_i} 和 $p_{a_{i'}}$ 修改，线段树上单点修改即可。

然后我们考虑如果此时两个值的状态变化了，即 $\in S \longleftrightarrow \notin S$ ，那 S 和 s 就会受到影响。线段树上的修改操作就是一段后缀 ± 1 。

此时后面的部分 v 也有可能出现集合状态变化的情况，但是我们只需要将最小的变化状态的 v 进行一次修复，后面的就都正常了。

Solution

考虑加入修改，一次 $a_i \rightarrow a_{i'}$ 至多会将 p_{a_i} 和 $p_{a_{i'}}$ 修改，线段树上单点修改即可。

然后我们考虑如果此时两个值的状态变化了，即 $\in S \longleftrightarrow \notin S$ ，那 S 和 s 就会受到影响。线段树上的修改操作就是一段后缀 ± 1 。

此时后面的部分 v 也有可能出现集合状态变化的情况，但是我们只需要将最小的变化状态的 v 进行一次修复，后面的就都正常了。而要找到这个 $v_{\min}$ ，我们需要维护 $v \in S$ 的 $v - s_v - \text{pre}_{p_v}$ 的最大值以及 $v \notin S$ 的权值的最小值，我们就能快速找到 $v_{\min}$ 了。

Solution

考虑加入修改，一次 $a_i \rightarrow a_{i'}$ 至多会将 p_{a_i} 和 $p_{a_{i'}}$ 修改，线段树上单点修改即可。

然后我们考虑如果此时两个值的状态变化了，即 $\in S \longleftrightarrow \notin S$ ，那 S 和 s 就会受到影响。线段树上的修改操作就是一段后缀 ± 1 。

此时后面的部分 v 也有可能出现集合状态变化的情况，但是我们只需要将最小的变化状态的 v 进行一次修复，后面的就都正常了。而要找到这个 $v_{\min}$ ，我们需要维护 $v \in S$ 的 $v - s_v - \text{pre}_{p_v}$ 的最大值以及 $v \notin S$ 的权值的最小值，我们就能快速找到 $v_{\min}$ 了。

单 $\log$ 过掉本题。

CF2182G
○○○○○○○

CF1572F
○○○○

LG P8861
○○○○○○○

LG P15397
○○○○○

LG P12485
○○○○

LG P8987
●○○○○

CF1685E
○○○○○○○

Outline

- CF2182G
- CF1572F
- LG P8861
- LG P15397
- LG P12485
- **LG P8987**
- CF1685E

Statement

P8987 [北大集训 2021] 简单数据结构

你有一个长度为 n 的序列 a ，下面你要进行 q 次修改或询问。

1. 给定 v ，将**所有** a_i 变为 $\min\{a_i, v\}$ 。
2. 将**所有** a_i 变为 $a_i + i$ 。
3. 给定 l, r ，询问 $\sum_{i=l}^r a_i$ 。

$$n, q \leq 2 \times 10^5, a_i, v \leq 10^{12}$$

Hints

1. 考虑初始 $a_i = +\infty$ 怎么做?

Hints

1. 考虑初始 $a_i = +\infty$ 怎么做?
2. 考虑如何求出 a_i 处理方式等价于 $+\infty$ 的时间点?

Solution

如果原来序列本身不降，那么每次 `chkmin` 和 $a_i \leftarrow a_i + i$ 的操作仍然能保证不降。

Solution

如果原来序列本身不降，那么每次 `chkmin` 和 $a_i \leftarrow a_i + i$ 的操作仍然能保证不降。

所以我们可以维护每个数 $+i$ 次数的同时，二分出来 `chkmin` 的范围（一段后缀）然后区间覆盖就行了。这个是易于通过线段树来解决的。

Solution

如果原来序列本身不降，那么每次 `chkmin` 和 $a_i \leftarrow a_i + i$ 的操作仍然能保证不降。

所以我们可以维护每个数 $+i$ 次数的同时，二分出来 `chkmin` 的范围（一段后缀）然后区间覆盖就行了。这个是易于通过线段树来解决的。

现在，我们就考虑推广这个做法。我们观察到，`chkmin` 操作真的很强，它会抹掉 a_i 原本的特征。思考一个 a_i 被实际上 `chkmin` 了（满足 $a_i > v$ ）之后，会怎么样？

Solution

如果原来序列本身不降，那么每次 `chkmin` 和 $a_i \leftarrow a_i + i$ 的操作仍然能保证不降。

所以我们可以维护每个数 $+i$ 次数的同时，二分出来 `chkmin` 的范围（一段后缀）然后区间覆盖就行了。这个是易于通过线段树来解决的。

现在，我们就考虑推广这个做法。我们观察到，`chkmin` 操作真的很强，它会抹掉 a_i 原本的特征。思考一个 a_i 被实际上 `chkmin` 了（满足 $a_i > v$ ）之后，会怎么样？显然，对于所有已经被抹掉的 a_i ，拎出来，这个序列就满足之后操作一直单调不降。

Solution

如果原来序列本身不降，那么每次 chkmin 和 $a_i \leftarrow a_i + i$ 的操作仍然能保证不降。

所以我们可以维护每个数 $+i$ 次数的同时，二分出来 chkmin 的范围（一段后缀）然后区间覆盖就行了。这个是易于通过线段树来解决的。

现在，我们就考虑推广这个做法。我们观察到， chkmin 操作真的很强，它会抹掉 a_i 原本的特征。思考一个 a_i 被实际上 chkmin 了（满足 $a_i > v$ ）之后，会怎么样？

显然，对于所有已经被抹掉的 a_i ，拎出来，这个序列就满足之后操作一直单调不降。

除了这些被抹掉之外的数呢？

Solution

如果原来序列本身不降，那么每次 chkmin 和 $a_i \leftarrow a_i + i$ 的操作仍然能保证不降。

所以我们可以维护每个数 $+i$ 次数的同时，二分出来 chkmin 的范围（一段后缀）然后区间覆盖就行了。这个是易于通过线段树来解决的。

现在，我们就考虑推广这个做法。我们观察到， chkmin 操作真的很强，它会抹掉 a_i 原本的特征。思考一个 a_i 被实际上 chkmin 了（满足 $a_i > v$ ）之后，会怎么样？

显然，对于所有已经被抹掉的 a_i ，拎出来，这个序列就满足之后操作一直单调不降。

除了这些被抹掉之外的数呢？我们发现没有被实际上 chkmin 的数，肯定是一直执行了 $+i$ 操作的，所以非常简单可以维护出来答案。

Solution

如果原来序列本身不降，那么每次 `chkmin` 和 $a_i \leftarrow a_i + i$ 的操作仍然能保证不降。

所以我们可以维护每个数 $+i$ 次数的同时，二分出来 `chkmin` 的范围（一段后缀）然后区间覆盖就行了。这个是易于通过线段树来解决的。

现在，我们就考虑推广这个做法。我们观察到，`chkmin` 操作真的很强，它会抹掉 a_i 原本的特征。思考一个 a_i 被实际上 `chkmin` 了（满足 $a_i > v$ ）之后，会怎么样？显然，对于所有已经被抹掉的 a_i ，拎出来，这个序列就满足之后操作一直单调不降。

除了这些被抹掉之外的数呢？我们发现没有被实际上 `chkmin` 的数，肯定是一直执行了 $+i$ 操作的，所以非常简单可以维护出来答案。

所以如果我们知道了什么时候一个数被实际上 `chkmin`，之前和之后分别在两颗线段树上维护，询问的时候两颗求和就能得到答案了。

Solution

如果原来序列本身不降，那么每次 chkmin 和 $a_i \leftarrow a_i + i$ 的操作仍然能保证不降。

所以我们可以维护每个数 $+i$ 次数的同时，二分出来 chkmin 的范围（一段后缀）然后区间覆盖就行了。这个是易于通过线段树来解决的。

现在，我们就考虑推广这个做法。我们观察到， chkmin 操作真的很强，它会抹掉 a_i 原本的特征。思考一个 a_i 被实际上 chkmin 了（满足 $a_i > v$ ）之后，会怎么样？显然，对于所有已经被抹掉的 a_i ，拎出来，这个序列就满足之后操作一直单调不降。

除了这些被抹掉之外的数呢？我们发现没有被实际上 chkmin 的数，肯定是一直执行了 $+i$ 操作的，所以非常简单可以维护出来答案。

所以如果我们知道了什么时候一个数被实际上 chkmin ，之前和之后分别在两颗线段树上维护，询问的时候两颗求和就能得到答案了。

所以问题的关键点转化为找到一个数被实际上 chkmin 的“切换”时间点。

Solution

我们定义一下时间点，每次取 $\min$ 的时候时间 $+1$.

我们去考虑一个整体二分的过程， $\text{search}(l, r, \text{idxs})$ 表示 idxs 这些数的“切换”时间点在 $[l, r]$ 内，递归边界就是 $l == r$ 时将 idxs 的“切换”时间点确定。

Solution

我们定义一下时间点，每次取 $\min$ 的时候时间 $+1$.

我们去考虑一个整体二分的过程， $\text{search}(l, r, \text{idxs})$ 表示 idxs 这些数的“切换”时间点 $\in [l, r]$ 内，递归边界就是 $l == r$ 时将 idxs 的“切换”时间点确定。

我们现在考虑，判断 idx 的“切换”时间点是否 $\leq \text{mid}$. 我们设在时间点 t 及之前，有 b_t 次整体 $+i$ 操作，在时间点 t 的时候与 v_i 取最小。

所以对于 a_i 的判断条件就是 $\exists j \in [1, \text{mid}], a_i + b_j i \geq v_j$.

Solution

我们定义一下时间点，每次取 $\min$ 的时候时间 $+1$.

我们去考虑一个整体二分的过程， $\text{search}(l, r, \text{idxs})$ 表示 idxs 这些数的“切换”时间点 $\in [l, r]$ 内，递归边界就是 $l == r$ 时将 idxs 的“切换”时间点确定。

我们现在考虑，判断 idx 的“切换”时间点是否 $\leq \text{mid}$. 我们设在时间点 t 及之前，有 b_t 次整体 $+i$ 操作，在时间点 t 的时候与 v_i 取最小。

所以对于 a_i 的判断条件就是 $\exists j \in [1, \text{mid}], a_i + b_j i \geq v_j$.

稍微改一下， $a_i \geq \min_{j=1}^{\text{mid}} \{-b_j i + v_j\}$.

Solution

我们定义一下时间点，每次取 $\min$ 的时候时间 $+1$ 。

我们去考虑一个整体二分的过程， $\text{search}(l, r, \text{idxs})$ 表示 idxs 这些数的“切换”时间点 $\in [l, r]$ 内，递归边界就是 $l == r$ 时将 idxs 的“切换”时间点确定。

我们现在考虑，判断 idx 的“切换”时间点是否 $\leq \text{mid}$ 。我们设在时间点 t 及之前，有 b_t 次整体 $+i$ 操作，在时间点 t 的时候与 v_i 取最小。

所以对于 a_i 的判断条件就是 $\exists j \in [1, \text{mid}], a_i + b_j i \geq v_j$ 。

稍微改一下， $a_i \geq \min_{j=1}^{\text{mid}} \{-b_j i + v_j\}$ 。

这个玩意，易于使用扫描线维护，即考虑每次往李超线段树加入一条线段 $-b_j x + v_j$ ，然后询问在横坐标 i 时的最小值。

所以直接分治的时候每层加一遍，所以又多一个 $\log$ 。无所谓了。能过。^_^

Outline

- CF2182G
- CF1572F
- LG P8861
- LG P15397
- LG P12485
- LG P8987
- **CF1685E**

Statement

CF1685E The Ultimate LIS Problem

给定一个由数字 1 到 $2n + 1$ 组成的排列 p .

你需要处理 q 次操作 (u, v) , 每次操作将交换 p_u 和 p_v .

每次操作后, 找出 p 的任意一个循环移位 $p_k, p_{k+1}, \dots, p_{2n+1}, p_1, p_2, \dots, p_{k-1}$, 使得该移位后的排列的 LIS 长度不超过 n , 输出 k 或者判断不存在这样的移位。

这里 $\text{LIS}(a)$ 表示序列 a 的最长严格递增子序列的长度。

$2 \leq n \leq 10^5, q \leq 10^5$.

Solution

本题 *35000. 不开玩笑。

Solution

本题 *35000. 不开玩笑。

令 $b_i = \begin{cases} -1 & \text{if } a_i < n+1 \\ 0 & \text{if } a_i = n+1 \\ 1 & \text{if } a_i > n+1 \end{cases}$, $sb_i = \sum_{j=1}^i b_j$, 我们考虑一个 $\forall sb_i \geq 0$ 的循环移位, 证明:

$$\forall sb_i \geq 0 \wedge |\text{LIS}| > n \implies n+1 \in \text{LIS}$$

Solution

本题 *35000. 不开玩笑。

令 $b_i = \begin{cases} -1 & \text{if } a_i < n+1 \\ 0 & \text{if } a_i = n+1 \\ 1 & \text{if } a_i > n+1 \end{cases}$, $sb_i = \sum_{j=1}^i b_j$, 我们考虑一个 $\forall sb_i \geq 0$ 的循环移位, 证明:

$$\forall sb_i \geq 0 \wedge |\text{LIS}| > n \implies n+1 \in \text{LIS}$$

考虑反证。由条件可得选中的 LIS 对应的 b_i 集合有 x 个 -1 和 y 个 1 , 且 $x + y \geq n + 1$. 考虑最优情况选中 b 最靠前的 x 个 -1 和最靠后的 y 个 1 .

Solution

本题 *35000. 不开玩笑。

令 $b_i = \begin{cases} -1 & \text{if } a_i < n+1 \\ 0 & \text{if } a_i = n+1 \\ 1 & \text{if } a_i > n+1 \end{cases}$, $sb_i = \sum_{j=1}^i b_j$, 我们考虑一个 $\forall sb_i \geq 0$ 的循环移位, 证明:

$$\forall sb_i \geq 0 \wedge |\text{LIS}| > n \implies n+1 \in \text{LIS}$$

考虑反证。由条件可得选中的 LIS 对应的 b_i 集合有 x 个 -1 和 y 个 1 , 且 $x + y \geq n + 1$. 考虑最优情况选中 b 最靠前的 x 个 -1 和最靠后的 y 个 1 .

$\because y \geq n - x + 1, \therefore$ 第 $n - (n - x + 1) + 1$ 个 1 对应的 a_i 一定在 LIS 中。

又 $\because \forall sb_i \geq 0, \therefore$ 第 x 个 1 一定出现在第 x 个 -1 之前。但是我们刚刚说明第 x 个 -1 和 1 都在 LIS 中, 由 LIS 性质可得第 x 个 1 一定在第 x 个 -1 之后。矛盾, ■.

Solution

本题 *35000. 不开玩笑。

令 $b_i = \begin{cases} -1 & \text{if } a_i < n+1 \\ 0 & \text{if } a_i = n+1 \\ 1 & \text{if } a_i > n+1 \end{cases}$, $sb_i = \sum_{j=1}^i b_j$, 我们考虑一个 $\forall sb_i \geq 0$ 的循环移位, 证明:

$$\forall sb_i \geq 0 \wedge |\text{LIS}| > n \implies n+1 \in \text{LIS}$$

考虑反证。由条件可得选中的 LIS 对应的 b_i 集合有 x 个 -1 和 y 个 1 , 且 $x+y \geq n+1$. 考虑最优情况选中 b 最靠前的 x 个 -1 和最靠后的 y 个 1 .

$\because y \geq n-x+1, \therefore$ 第 $n-(n-x+1)+1$ 个 1 对应的 a_i 一定在 LIS 中。

又 $\because \forall sb_i \geq 0, \therefore$ 第 x 个 1 一定出现在第 x 个 -1 之前。但是我们刚刚说明第 x 个 -1 和 1 都在 LIS 中, 由 LIS 性质可得第 x 个 1 一定在第 x 个 -1 之后。矛盾, ■.

所以我们证明了这样的移位 LIS 对应的 b_i 一定形如 $\{\underbrace{-1, \dots, -1}_x, 0, \underbrace{1, \dots, 1}_y\}$

满足 $x+y \geq n \implies y \geq n-x$. 后面至少有 $n-x$ 个 $1 \implies$ 前面至多 x 个 1 .

Solution

后面至少有 $n - x$ 个 1 $\implies$ 前面至多 x 个 1. 但是前面有 x 个 -1 , 又根据 $\forall sb_i \geq 0$ 我们能够知道前面至少有 x 个 1, 所以事实上就是前面有 x 个 -1 和 1, 后面有 $n - x$ 个 -1 和 1. 我们就得到了优美的 LIS 形式。

然后我们再考虑将这种优美的移位修正到 $n + 1$ 为首项和 $n + 1$ 为末项。此时由刚刚证得的结论, 知道 LIS 分别为 $\{n + 1, n + 2, \dots, 2n + 1\}$ 和 $\{1, 2, \dots, n + 1\}$.

现在我们尝试证明, 只要满足 $\exists |\text{LIS}| > n, \forall sb_i \geq 0$ 且 $n + 1$ 移位至最前和最后的时候能得到上述 LIS, 那么任何一种循环移位都无解。

构造证明: 只需要取 $b_i = 0$ 的位置、该位置前面所有的 -1 和后面所有的 1 的位置 (这些位置天然递增), 构成的 IS 长度就是 $> n$ 的。

Solution

现在梳理一下流程：

1. 判断 $n + 1$ 在最前面的移位是否 $\forall sb_i \geq 0$. 否：输出一个 $\forall sb_i \geq 0$ 的移位。

Solution

现在梳理一下流程：

1. 判断 $n + 1$ 在最前面的移位是否 $\forall sb_i \geq 0$. 否：输出一个 $\forall sb_i \geq 0$ 的移位。
考虑反证法证明构造正确性，如果其 LIS $> n$ 必然包含 $n + 1$, 则可以移位之后说明 $n + 1$ 在最前面的时候也有 $\forall sb_i \geq 0$, 矛盾，■.

Solution

现在梳理一下流程：

1. 判断 $n + 1$ 在最前面的移位是否 $\forall sb_i \geq 0$. 否：输出一个 $\forall sb_i \geq 0$ 的移位。
考虑反证法证明构造正确性，如果其 $LIS > n$ 必然包含 $n + 1$, 则可以移位之后说明 $n + 1$ 在最前面的时候也有 $\forall sb_i \geq 0$, 矛盾，■.
2. 判断 $n + 1$ 移位到首的时候 LIS 是否为 $n + 1 \sim 2n + 1$. 否：输出当前移位。
3. 判断 $n + 1$ 移位到末的时候 LIS 是否为 $1 \sim n + 1$. 否：输出当前移位。

Solution

现在梳理一下流程：

1. 判断 $n + 1$ 在最前面的移位是否 $\forall sb_i \geq 0$. 否：输出一个 $\forall sb_i \geq 0$ 的移位。
考虑反证法证明构造正确性，如果其 LIS $> n$ 必然包含 $n + 1$, 则可以移位之后说明 $n + 1$ 在最前面的时候也有 $\forall sb_i \geq 0$, 矛盾，■.
2. 判断 $n + 1$ 移位到首的时候 LIS 是否为 $n + 1 \sim 2n + 1$. 否：输出当前移位。
3. 判断 $n + 1$ 移位到末的时候 LIS 是否为 $1 \sim n + 1$. 否：输出当前移位。

第 1 步可以破坏成链，然后判断 sb_1 是否为 $sb_{1 \sim 2n+1}$ 中的最小值即可。构造直接取全局最小的 sb_i 点就行了，后面肯定有保证 $sb_i \geq 0$ 的。

Solution

现在梳理一下流程：

1. 判断 $n + 1$ 在最前面的移位是否 $\forall sb_i \geq 0$. 否：输出一个 $\forall sb_i \geq 0$ 的移位。
考虑反证法证明构造正确性，如果其 LIS $> n$ 必然包含 $n + 1$, 则可以移位之后说明 $n + 1$ 在最前面的时候也有 $\forall sb_i \geq 0$, 矛盾，■.
2. 判断 $n + 1$ 移位到首的时候 LIS 是否为 $n + 1 \sim 2n + 1$. 否：输出当前移位。
3. 判断 $n + 1$ 移位到末的时候 LIS 是否为 $1 \sim n + 1$. 否：输出当前移位。

第 1 步可以破坏成链，然后判断 sb_1 是否为 $sb_{1 \sim 2n+1}$ 中的最小值即可。构造直接取全局最小的 sb_i 点就行了，后面肯定有保证 $sb_i \geq 0$ 的。

第 2、3 步可以维护 $1 \sim n + 1$ 和 $n + 1 \sim 2n + 1$ 两个环，每两个点之间距离的和。如果距离之和刚好等于单环长 $2n + 1$ ，那么就是满的 LIS。

此题终了。

Thank You!